BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 051 · Lesson 051 · Linked Lists: Nodes and References

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. How does Queue<T> remove items?

Q2. How does a stack differ from a queue?

Q3. What is an object reference?

Q4. 019

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

node — Value and link information.

Next — Reference to following node.

Previous — Reference to preceding node.

head — First list node.

New learning

Discover — A linked-list node holds a value plus one or more links to neighbouring nodes, rather than occupying an indexed fixed-size array slot.

Trace — C# LinkedList<T> is doubly linked: nodes expose Previous and Next; AddAfter inserts relative to an existing node.

Evaluate — To insert B between A and C, change logical links to A↔B↔C. The operation differs from shifting array elements.

Worked example

Trace AddAfter to insert X after A in A→B.

1. Explain the trace — New order A,X,B. The inserted node links between A and B.

We Do · Practise with guidance

Remove the middle node from a different list P→Q→R.

```
var l=new LinkedList<string>(new[]{"P","Q","R"});
l.Remove("Q");
```

Explain the trace

P,R remain. After deletion P and R are adjacent logical neighbours.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Draw the logical next-links in HEAD→3→6→NULL after inserting 4 after 3. [6]

Question 2

Write C# using LinkedList<int> to create 2,4 then insert 3 between them. [6]

Question 3

Compare middle insertion in linked list with a C# List<int>, including locating position and access. [6]

Exit Ticket · Answer before checking

A linked list always offers O(1) indexed access to its nth node.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. First-in-first-out, from front.

Q2. LIFO versus FIFO.

Q3. Value identifying an object instance.

Q4. A linked list has HEAD→5→9→NULL. Insert 7 between 5 and 9. Give new traversal.

You Do pages · Independent questions

Q1. 3.next becomes 4 [2], 4.next=6 [2], final HEAD→3→4→6→NULL [2].

Q2. var l=new LinkedList<int>(); [1]; l.AddLast(2); l.AddLast(4); [2]; l.AddAfter(l.First!,3); [3].

Q3. List indexed access O(1) [2], insertion shifts subsequent values O(n) [1]; linked-list insertion with node reference O(1) links [2], finding node may take O(n) [1].

Exit Ticket · Model answer

False. Traversal is generally required.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

BLACK PEN Answer all questions first.

GREEN PEN Then: tick and rewrite mistakes.